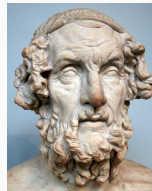


Formalising Homeric Prosody in Agda

Orestis Melkonian, Konstantinos “Kokos” Kogkalidis

9 September 2026, Wadler70, Edinburgh



*Marble portrait of Phil Wadler,
British Museum*

How it all started

- Kokos approaches me with a challenge:
“let’s formalise the rules of Homeric prosody”

HOMERIC GREEK

A BOOK FOR BEGINNERS

BY

CLYDE PHARR

PH.D., YALE UNIVERSITY

PROFESSOR OF GREEK IN SOUTHWESTERN
PRESBYTERIAN UNIVERSITY

To love Homer, as Steele said about loving a
fair lady of quality, “is a liberal education.”

— ANDREW LANG

D. C. HEATH & CO., PUBLISHERS

BOSTON

NEW YORK

CHICAGO

How it all started

- Kokos approaches me with a **challenge**:
“let’s formalise the rules of Homeric prosody”
- always keen on such hobby-/side-projects

MOTTO: **breadth** **VS** depth

HOMERIC GREEK

A BOOK FOR BEGINNERS

BY

CLYDE PHARR

PH.D., YALE UNIVERSITY

PROFESSOR OF GREEK IN SOUTHWESTERN
PRESBYTERIAN UNIVERSITY

To love Homer, as Steele said about loving a
fair lady of quality, “is a liberal education.”

— ANDREW LANG

D. C. HEATH & CO., PUBLISHERS

BOSTON

NEW YORK

CHICAGO

How it all started

- Kokos approaches me with a challenge:

“let’s formalise the rules of Homeric prosody”

Nominal techniques as an Agda library

Murdoch J. Gabbay¹ and Orestis Melkonian²

¹ Heriot-Watt University, Scotland

² University of Edinburgh, Scotland

Introduction Nominal techniques [8] provide a mathematically principled approach to dealing with names and variable binding in programming languages. However, integrating these ideas in a practical and widespread toolchain has been slow, and we perceive a chicken-and-egg problem: there are no users for nominal techniques, because nobody has implemented them, and nobody implements them because there are no users. This is a pity, but it leaves a positive opportunity to set up a virtuous circle of broader understanding, adoption, and application of this beautiful technology.

This paper explores an attempt to make nominal techniques accessible as a library in the Agda proof assistant and programming language [9], which can be viewed as a port of the first

HOMERIC GREEK

A BOOK FOR BEGINNERS

BY

CLYDE PHARR

PH.D., YALE UNIVERSITY

PROFESSOR OF GREEK IN SOUTHWESTERN

PRESBYTERIAN UNIVERSITY

To love Homer, as Steele said about loving a fair lady of quality, “is a liberal education.”

— ANDREW LANG

D. C. HEATH & CO., PUBLISHERS

BOSTON

NEW YORK

CHICAGO



Prosody for the type theorist

Prosody in one slide

- a **verse** is a sequence of **syllables** (the Iliad has 15 691 verses)
- every syllable carries a **quantity**: long (–) or short (⊔)
- **prosody** = the pattern of quantities a verse realises

– **1190.** For metrical purposes all vowels and syllables of Greek words may be divided into long and short.

1191. The rhythm of Greek verse is based upon the regular succession of long and short syllables.

Prosody in one slide

- a **verse** is a sequence of **syllables** (the Iliad has 15 691 verses)
- every syllable carries a **quantity**: long (—) or short (υ)
- **prosody** = the pattern of quantities a verse realises

— **1190.** For metrical purposes all vowels and syllables of Greek words may be divided into long and short.

1191. The rhythm of Greek verse is based upon the regular succession of long and short syllables.

The very first verse of the Iliad:

— υ υ — υ υ — — — υ υ — υ υ — —
μῆ νιν ἄ ει δε θε ἄ Πη λη ῖ ἄ δεω Ἄ χι λῆ ος

"Sing, goddess, the wrath of Achilles son of Peleus"

Dactylic hexameter

Homeric epics are written in **dactylic hexameter**:

1179. The metre of the Homeric poems is the *dactylic* (sometimes called the *heroic*) hexameter, the most common of all Greek verse.

1180. There are six feet to the verse, the first five being either *dactyls* (that is, one long followed by two shorts — ∪ ∪), or its equivalent, the *spondee* (that is, two longs — —). The sixth foot is always a spondee.

1 verse = 6 **feet**, where foot = **dactyl** (— ∪ ∪) or **spondee** (— —):

—	∪	∪		—	∪	∪		—	—		—	∪	∪		—	∪	∪		—	—
μῆ	νιν	ἄ		εἰ	δε	θε		ἄ	Πη		λη	ϊ	ά		δεω	Ἄ	χι		λη	ος

Scansion: the inverse problem

Scansion = inferring the meter from the written verse alone

- quantities are only *partially* determined by spelling
- the rest is constrained by accents, neighbouring syllables, ... and the meter itself

Scansion: the inverse problem

Scansion = inferring the meter from the written verse alone

- quantities are only *partially* determined by spelling
- the rest is constrained by accents, neighbouring syllables, ... and the meter itself

Existing tools are lacking in rigour:

- automata & ad-hoc systems in the digital humanities
- the most popular site (hypotactic.com) provides scansion, but no explanations
- no resource that explains ALL of the Iliad

Iliagda

Reading Pharr: a constructivist's nightmare

_ **1190.** For metrical purposes all vowels and syllables of Greek words may be divided into long and short.

1191. The rhythm of Greek verse is based upon the regular succession of long and short syllables.

- unintuitive order: this is the **last page!** of the book
- incoherent, conflating concerns, inconsistent naming

Reading Pharr: a constructivist's nightmare

525. Sometimes a short vowel followed by λ , μ , ν or ρ (occasionally σ or φ) forms a syllable long by position, in which case these consonants seem to have been doubled in pronunciation, and are sometimes so written.

- too many modal statements

Reading Pharr: a constructivist's nightmare

519. **Quantity.** — The vowels η and ω are always long; ϵ and o are always short, while α , ι , and υ are sometimes long and sometimes short, and hence are called *doubtful* vowels.

520. When the doubtful vowels are long in this text, it will be indicated (except in the direct quotations from Homer) by their having the mark ($\bar{}$) placed over them, as $\theta\epsilon\bar{\alpha}$ goddess. This mark will not be placed over vowels having the circumflex accent (534), as they are always long (537).

quantities of **doubtful** vowels are marked everywhere... **except in Homer** (our input!)

Reading Pharr: a constructivist's nightmare

1164. Exceptions to these rules are to be found only in the cases of the diphthongs *αι* and *οι*, when final, which are then considered short (except in the optative and *οἴκοι*) for the purpose of accent but must be counted long when marking the feet of the verse (547).

1165. Apparent exceptions to these rules are to be found in the case of certain classes of compounds, as *οὔτε, μήτε, οὔτις, μήτις, ἤδε, οἶδε, αἶδε, τοῦσδε, τᾶσδε*, etc., where the primary form is accented without considering the following enclitic as an integral part of the word.

1166. Most exceptions to the rules of quantity are only apparent.

- exceptions, as well as “apparent” exceptions to the exceptions
- reminiscent of **defeasible/default** logic, non-monotonic reasoning, etc.

Reading Pharr: a constructivist's nightmare

526. One of the consonants which make a syllable long by position (particularly φ , occasionally σ) is sometimes lost, but in many of these cases the φ was probably pronounced in Homeric times.

- lost letters, despair...

Reading Pharr: a constructivist's nightmare

70. The first syllable of $\mu\acute{\eta}\nu\upsilon$ must be long, having a long vowel, 522. The accent shows that the second syllable must be short, 545, 1160. Then, having one long and one short, the next syllable ($\acute{\alpha}$ -) must be short, as when there is one long followed by a short, there must be another short to complete the foot, 1169.

The next syllable ($-\epsilon$ -) is long, being a diphthong, 521–522.

The next syllable ($-\delta\epsilon$) is short, having a short vowel followed by a single consonant.

The next syllable ($\theta\epsilon$ -) is short also, having a short vowel followed by another vowel.

As this completes this foot, the final syllable ($-d$) of $\theta\epsilon d$ must be long, as it is the first syllable of the next foot, 1171.

The next syllable ($\Pi\eta$ -) is long, having a long vowel, 522.

The next syllable ($-\lambda\eta$ -) is long, having a long vowel, 522.

The next vowel ($-\iota$) is short. See the vocabulary.

Then the next syllable ($-d$ -) must be short to complete the foot.

The next vowel ($-\epsilon$ -) is short, but as the first syllable of a foot must be long, 1171, the $-\delta\epsilon\omega$ is pronounced as one long syllable, by synizesis, 586.

The next vowel ('A -) is short. See the vocabulary.

Then the next syllable ($-\chi\iota$ -) must be short to complete the foot.

The next syllable ($-\lambda\acute{\eta}$ -) is long, having a long vowel.

The final syllable ($-\omega$) is short, having a short vowel, but must be marked long here, as the final syllable is always long, owing to the pause in the verse, 1184.

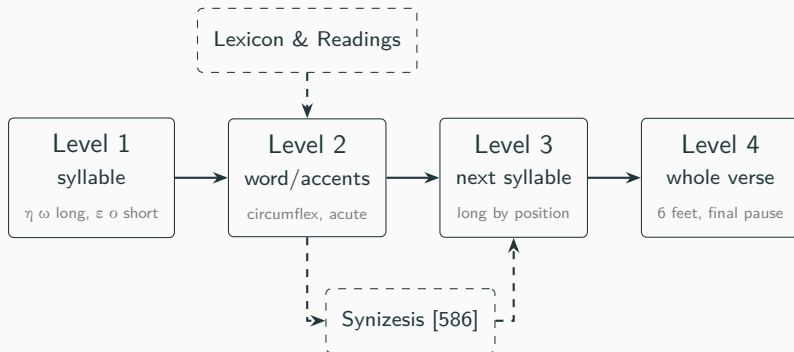
Therefore the metrical scheme of this verse is:

— — | — — | — — | — — | — — | — —

- few explanations: just a single page out of 400

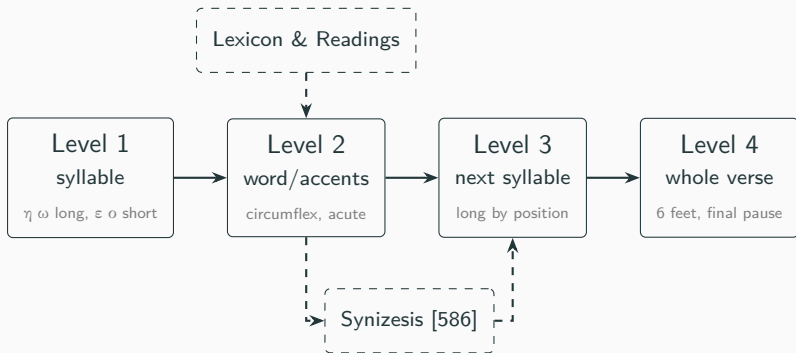
Architecture: declarative $\rightsquigarrow$ algorithmic

Stratify the rules by **how much context** they may consult:



Architecture: declarative $\rightsquigarrow$ algorithmic

Stratify the rules by **how much context** they may consult:



- each level emits **partial knowledge** about quantities
- Level 4 **reifies**: only fully determined assignments form feet

Morphology

```
data Letter : Type where
  -- vowels
  A α Ἀ á Ā ǎ Ă ă Å å Æ æ Ą ą : Letter
  ⋮
  -- consonants
  B β Γ γ Δ δ Ζ ζ Θ θ Κ κ Λ λ Μ μ Ν ν Ξ ξ : Letter
  ⋮
  -- special symbols
  ' {- apostrophe -} : Letter
  Ϝ {- digamma      -} : Letter
```

Morphology

Syllable = List⁺ Letter

Syllables = Vec Syllable

```
data Word : ℕ {- syllables -} → Type where  
  word : { _ : auto : n ≠ 0 } → Syllables n → Word n
```

```
data Words : ℕ → Type where  
  [] : Words 0  
  _::_ : Word n → Words n' → Words (n + n')
```

- **words** indexed by their number of syllables
- a **verse** is just **Words** n : concatenation tracks n at the type level

Quantities & meters

```
data Quantity : Type where
  · {- short -} : Quantity
  — {- long -} : Quantity
```

```
data Foot : (n : ℕ) → Vec Quantity n → Type where
  ... {- dactyl -} : Foot 3 (— :: · :: · :: [])
  — — {- spondee -} : Foot 2 (— :: — :: [])
```

```
data Meter : ℕ → ℕ {- feet -} → Type where
  mkPM : (fs : Feet) → Meter (Σ1 fs) (length fs)
```

```
Hexameter : ℕ → Type
Hexameter n = Meter n 6
```

- **feet** are indexed by their quantities
- **hexameters**: 6 feet over n syllables

Everything is a compliance relation

```
record _compliesWith_ (A B : Type) : Type1 where
```

```
  infix 0 _~_
```

```
  field _~_ : A → B → Type
```

```
_⊄_ : A → B → Type
```

```
_⊄_ = ¬_ ∘2 _~_
```

```
NonDerivable : A → Type
```

```
NonDerivable a = ∀ b → a ⊄ b
```

- one interface, instantiated at **every** level:
 - syllable ~ quantity, word ~ quantities, verse ~ hexameter, ...
- **NonDerivable**: constructive “there exist no possible derivation”

Level 1: inherent quantity

–Vowel •Vowel Doubtful : Pred₀ Letter

[illegible]

- Vowel = $_e$ [E ; ε ; Ē ; Ĕ ; Ė ; Ğ ; Ĥ ; Ħ ; Ĩ ; Ĵ ; Ķ ; Ĳ ; ĸ ; Ĺ]
 - ◊ [O ; o ; Ō ; Ŏ ; Ȯ ; Ȭ ; Ȯ ; Ȭ ; Ȯ ; Ȭ ; Ȯ ; Ȭ]
 - ◊ [Ä ; ä ; Ī ; ĩ ; Ÿ ; ü]

$$\text{Doubtful} = (\neg_ \circ \text{-Vowel}) \cap^1 (\neg_ \circ \cdot \text{Vowel})$$

the constructive reading of [§519]: **doubtful** = neither derivably long nor derivably short

Level 1: inherent quantity

Sy-Q : Syllable -compliesWith- Quantity

Sy-Q . $\sim$ _ = $\sim$ ' _

where

data $\sim$ ' _ : Syllable $\rightarrow$ Quantity $\rightarrow$ Type where

longByNature :

(Any $\times$ Diphthong sy
 $\sqcup$ Any -Vowel sy
 $\sqcup$ Any HasCircumflex sy)

sy $\sim$ ' _

shortByNature :

$\forall$ (v $\in$: Any $\cdot$ Vowel sy) $\rightarrow$
 • vowels sy $\equiv$ 1

sy $\sim$ ' .

Modal? Just three-valued works!

```
data Flat (A : Type) : Type where
  single : A → Flat A
  none    : Flat A
  all     : Flat A
```

```
Quantities : ℕ → Type
Quantities = Vec (Flat Quantity)
```

```
_⊗₁_ : Op₂ $ Flat Quantity
_⊗₁_ = λ where
  (single _) (single q) → single q -- RIGHT BIASED
  (single q) none      → single q
  (single _) all       → all
  none      mq         → mq
  all      mq         → mq
```

```
_⊗_ : Op₂ $ Quantities n
_⊗_ = V.zipWith _⊗₁_
```

Modal? Just three-valued works!

```
data Flat (A : Type) : Type where
  single : A → Flat A
  none    : Flat A
  all     : Flat A
```

```
Quantities : ℕ → Type
Quantities = Vec (Flat Quantity)
```

```
_⊗_ : Op₂ $ Flat Quantity
_⊗_ = λ where
  (single _) (single q) → single q -- RIGHT BIASED
  (single q) none      → single q
  (single _) all       → all
  none      mq         → mq
  all      mq         → mq
```

```
_⊗_ : Op₂ $ Quantities n
_⊗_ = V.zipWith _⊗_
```

- **none** = no opinion, **single** = certain, **all** = *ambivalent* (long **and** short)
- levels combine with the **right-biased** **_⊗_**: later, more informed levels win

Level 1: inherent quantity

Sy-MQ : Syllable -compliesWith- Flat Quantity

Sy-MQ . $\sim$ = $\sim'$

where

data $\sim'$: Syllable $\rightarrow$ Flat Quantity $\rightarrow$ Type where

byNature :

sy $\sim$ q

sy $\sim'$ single q

doubtful :

NonDerivable {B = Quantity} sy

sy $\sim'$ none

doubtful vowels leave the judgement **partial**: on to the next level

Level 2: accents constrain the last three syllables

```
data _≡_~%'_ : Quantities n → Syllables n → Op₁ (Quantities n) → Type where
```

```
-- The vowel of the ultima in every  
-- word having the circumflex on  
-- the penult is short (545).
```

```
[1160] :
```

```
InPenult (Any HasCircumflex) sys
```

```
mqs ⊢ sys ~%' (_:=n single .)
```

```
-- If a long penult has the acute  
-- accent, then the ultima must  
-- be long also.
```

```
[1161] :
```

- InPenult (_≡ single -) mqs
 - InPenult (Any HasAcute) sys
-

```
mqs ⊢ sys ~%' (_:=n single -)
```

```
-- If the ultima is short and the  
-- penult has the acute accent, then  
-- the penult must be short also.
```

```
[1162] :
```

- InUlt (_≡ single .) mqs
 - InPenult (_≠ single -) mqs
 - InPenult (Any HasAcute) sys
-

```
mqs ⊢ sys ~%' (_:=n-1 single .)
```

```
-- If the antepenult has the accent,  
-- the vowel of the ultima must be  
-- short (544).
```

```
[1163] :
```

```
InAntepenult (Any HasAccent) sys
```

```
mqs ⊢ sys ~%' (_:=n single .)
```

Level 2: default logic via stratification

```
data _⊢_~%_ : Quantities n → Syllables n → Op1 (Quantities n) → Type where
```

```
[1164] :  
  EndsInFinalDiphthong sys
```

```
mqs ⊢ sys ~% id
```

```
[574] :  
  ApparentException sys
```

```
mqs ⊢ sys ~% id
```

```
-- (575/583) Elision  
-- has taken place.
```

```
[575] :  
  EndsInApostrophe sys
```

```
mqs ⊢ sys ~% id
```

```
fromBelow : ∀ {f} →
```

- ¬ EndsInFinalDiphthong sys
 - ¬ ApparentException sys
 - ¬ EndsInApostrophe sys
 - SingleAccents sys
 - mqs ⊢ sys ~%' f
-

```
mqs ⊢ sys ~% f
```

```
noop :
```

- (¬ SingleAccents sys)
- ⊔ (∀ {f} → ¬ (mqs ⊢ sys ~%' f))
-

```
mqs ⊢ sys ~% id
```

Level 3: peeking at the next syllable

```
data Context : Type where
  ∅      : Context
  inner : Syllable → Context
  outer : ∃ Word → Context
```

the context may cross a word boundary (**outer**): quantity depends on **verse** structure

Level 3: peeking at the next syllable

```
module _ (ctx : Flat Quantity × Context) (let mq , next = ctx) where
```

```
:
```

```
data ~*_ : Syllable → Quantity → Type where
```

```
-- long by position
```

```
[522] :
```

```
(v ∈ : Any Vowel sy) →
```

- FollowedBy (StartsWithDoubleConsonant
 U¹ StartsWithTwoConsonants) v ∈

```
sy ~* -
```

```
-- (572) long vowels may be shortened
```

```
-- before another vowel
```

```
[1173] :
```

```
(v ∈ : Any Vowel sy) →
```

- mq ≡ single -
- LastAny v ∈
- FollowedBy StartsWithVowel v ∈

```
sy ~* q
```

```
-- mutes followed by liquids make a *common* syllable
```

```
[524] :
```

```
(v ∈ : Any Vowel sy) →
```

- mq ≡ single .
- FollowedByInner MuteThenLiquid v ∈
- ⊕ FollowedByOuter MuteThenLiquid v ∈

```
sy ~* q
```

Level 3: peeking at the next syllable

```
data ~?_ : Syllable → Flat Quantity → Type where
```

```
ambiguous :
```

```
( $\forall q \rightarrow \text{sy } \not\star q$ )
```

```
sy ~? none
```

```
ambivalent :
```

- $\text{sy } \sim\star -$
- $\text{sy } \sim\star \cdot$

```
sy ~? all
```

```
certain :
```

- $\text{sy } \sim\star q$
- $\text{sy } \not\star (! q)$

```
sy ~? single q
```

three-valued verdicts: **ambiguous**, **ambivalent** (common syllables), **certain**

Level 4: feet, at last

```
-- (1180) There are six feet to the verse...
data _~'_ : Words n × Quantities n → Hexameter n → Type where
  reify : {mqs : Quantities n} → let mkLastLong = _=n⟨ Hex>0 hm ⟩ single - in
    -- (1184) The last syllable of a verse is considered long (due to pause)
    • mkLastLong mqs -mask*- qs
    • ws , qs ~m hm
```

(ws , mqs) ~' hm

Level 4: feet, at last

-- (1180) There are six feet to the verse.

data ~' _ : Words n × Quantities n → Hexameter

reify : {mqs : Quantities n} → let mkLastLon

-- (1184) The last syllable of a verse is considered long (due to pause)

- mkLastLong mqs -masks*- qs

- ws , qs ~^m hm

(ws , mqs) ~' hm

```
data _-masks-_ {X : Type} : Flat X → X → Type where
```

```
  none  : none    -masks- x
```

```
  all   : all     -masks- x
```

```
  single : single x -masks- x
```

```
_-masks*-_ : Vec (Flat X) n → Vec X n → Type
```

```
_-masks*-_ = VPointwise _-masks-_
```

Level 4: feet, at last

```
data ~m : Words n × Vec Quantity n → Meter n m → Type where
```

```
[ ] :
```

```
[ ] , [ ] ~m mkPM [ ]
```

```
spondee :
```

```
dropSys 2 ws , qs ~m pm
```

```
ws , - :: - :: qs ~m — ::p pm
```

```
dactyl :
```

```
dropSys 3 ws , qs ~m pm
```

```
ws , - :: · :: · :: qs ~m — · ::p pm
```

Level 4: feet, at last

-- lengthen by thesis

[1168] : let sy' = firstSy ws in

- EndsWith [Vowel ; Consonant] (toList sy)
- BeginsWith [Vowel] (toList sy')
- word [sy] :: ws , - :: qs ~^m pm

word [sy] :: ws , . :: qs ~^m pm

-- Whenever a word ends within a foot, it is called *caesura*. (1185)

[1167/1a] :

word [sy] :: ws , - :: qs ~^m pm

word [sy] :: ws , . :: qs ~^m pm

-- Whenever the end of a word coincides with the end of a foot, it is called *diaeresis*. (1188)

[1167/1b] :

- Split 2 ws
- dropSys 2 ws , qs ~^m pm

ws , - :: . :: qs ~^m (— :: ^pm pm)

The whole pipeline in one rule

`data _~'_ : Words  $n$  → Hexameter  $n'$  → Type where`

`↳_>_>_>_>_>_ :  $\forall \{ws \text{ } ws^T : \text{Words } n\} \{sys' : \text{Syllables } n'\} \{hm : \text{Hexameter } n'\}$`

`→  $ws \text{ } \text{-reads- } ws^T$`

`→  $ws^T \sim^2 mqs_2$`

`-- [586] synizesis -----`

`→ ( $syn : \text{unwords } ws^T \text{ } \text{-synizizes*- } sys'$ ) →`

`let`

`$ws' = \text{synizizeWords } ws^T \text{ } syn$`

`$mqs_2' = \text{synizize } syn \text{ } mqs_2$`

`in`

`• ( $ws' , mqs_2'$ )  $\sim^3 mqs_3$`

`• ( $ws' , mqs_2' \otimes mqs_3$ )  $\sim^4 hm$`

`$ws \sim' hm$`

read ► lexicon & accents >>> synizesis >>> context >>> meter

The whole pipeline in one rule

$\text{data_}\sim\text{'_'} : \text{Words } n \rightarrow \text{Hexameter } n' \rightarrow \text{Type where}$

$$_ \triangleright _ \langle _ \rangle \gg _ : \forall \{ws \ ws^T : \text{Words } n\} \{sys' : \text{Syllables } n'\} \{hm : \text{Hexameter } n'\}$$

→ WS -reads- WS^T

$$\rightarrow WS^T \sim^2 mqs_2$$

```
-- [586] synizesis -----
```

$$\rightarrow (\text{syn} : \text{unwords } \text{ws}^T \text{ -synzizes* - sys}') \rightarrow$$

let

$$WS' = \text{synthesizeWords } WS^T \text{ syn}$$
$$mqs_2' = \text{synimize syn } mqs_2$$

in

- $(ws', mqs_2') \sim^3 mqs_3$

- $(ws', mqs_2' \otimes mqs_3) \sim^4 hr$

WS ~' hm

```
-- when all else fails, merge two adjacent vowel syllables and rescan (586)
```

```
data _synsizes*_- : Syllables n → Syllables n' → Type
```

where

$$\begin{bmatrix} \cdot \\ \cdot \end{bmatrix} : \begin{bmatrix} \cdot \\ \cdot \end{bmatrix} \sim \begin{bmatrix} \cdot \\ \cdot \end{bmatrix}$$
$$\vdash :: \vdash : \forall sy \rightarrow sys \sim sys' \rightarrow (sy :: sys) \sim (sy :: sys')$$
$$\vdash_{\text{M}} : \text{Merge } sy \ sy' \rightarrow sys \sim sys' \rightarrow (sy :: sy' :: sys) \sim (sy \hat{\ } sy' :: sys')$$

read ► lexicon & accents ►► synthesis ►► context ►► meter



The marriage of heaven & hell: lexicon & readings

- pure rules meet reality: some quantities are just facts of the language

```
lexicon : List Entry
```

```
lexicon
```

```
= mkEntry [ ᾱ ; μ ; φ ; ι ] (prefix 1) .  
:: mkEntry [ ὀ ; ξ ; ù ] (exact $ from-end 0) .  
:: mkEntry [ ἔ ; ρ ; γ ; α ] (exact $ from-end 0) .  
⋮
```

```
readings : List Reading
```

```
readings
```

```
= unwritten [ ἔ ; δ ; ε ; ι ; σ ; ε ; ν ] F 1 1  
:: unwritten [ Β ; ο ; ρ ; έ ; η ] ρ 0 2  
:: unwritten [ φ ; ι ; λ ; ο ; μ ; ε ; ι ; δ ; ῆ ; ς ] μ 1 2  
⋮
```

Decidability proofs as decision procedures

```
record Enumeration ( _ ~ _ : A → B → Type ) : Type where
  field
    allBs      : A → List B
    sound      : ∀ {a b} → b ∈ allBs a → a ~ b
    complete   : ∀ {a b} → a ~ b → b ∈ allBs a
```

```
allHexameters :
  (smqs : Words n × Quantities n) →
  ∃ λ (hms : List (Hexameter n)) →
    (∀ {hm} → hm ∈ hms → smqs ~ hm)
  × (∀ {hm} → smqs ~ hm → hm ∈ hms)
```

Decidability proofs as decision procedures

```
record Enumeration (_~_ : A → B → Type) : Type where
  field
    allBs      : A → List B
    sound      : ∀ {a b} → b ∈ allBs a → a ~ b
    complete   : ∀ {a b} → a ~ b → b ∈ allBs a
```

```
allHexameters :
  (smqs : Words n × Quantities n) →
  ∃ λ (hms : List (Hexameter n)) →
    (∀ {hm} → hm ∈ hms → smqs ~ hm)
  × (∀ {hm} → smqs ~ hm → hm ∈ hms)
```

scansion = running the spec; derivations then pretty-printed as explanations

Chasing the dragon

- extracted to Haskell & compiled; minor optimisations to keep proof search feasible

How many verses can we scan?

1 $\rightsquigarrow$

Chasing the dragon

- extracted to Haskell & compiled; minor optimisations to keep proof search feasible

How many verses can we scan?

1 $\rightsquigarrow$ 10 $\rightsquigarrow$

Chasing the dragon

- extracted to Haskell & compiled; minor optimisations to keep proof search feasible

How many verses can we scan?

1 $\rightsquigarrow$ 10 $\rightsquigarrow$ 20 $\rightsquigarrow$

Chasing the dragon

- extracted to Haskell & compiled; minor optimisations to keep proof search feasible

How many verses can we scan?

1 $\rightsquigarrow$ 10 $\rightsquigarrow$ 20 $\rightsquigarrow$ 50 $\rightsquigarrow$

Chasing the dragon

- extracted to Haskell & compiled; minor optimisations to keep proof search feasible

How many verses can we scan?

1 $\rightsquigarrow$ 10 $\rightsquigarrow$ 20 $\rightsquigarrow$ 50 $\rightsquigarrow$ 15 679 / 15 691 (>99.9%)

Chasing the dragon

- extracted to Haskell & compiled; minor optimisations to keep proof search feasible

How many verses can we scan?

1 $\rightsquigarrow$ 10 $\rightsquigarrow$ 20 $\rightsquigarrow$ 50 $\rightsquigarrow$ 15 679 / 15 691 (>99.9%)

- every new batch of verses falsified our current reading of Pharr
- corpus as the ground truth: rules were fixed, modified, re-stratified

What about the remaining 12 verses?

- not logical gaps: verses flagged as metrically anomalous in the literature

What about the remaining 12 verses?

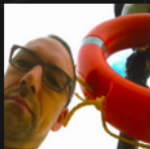
- not logical gaps: verses flagged as metrically anomalous in the literature
- e.g. Δ 155: φίλε κασίγνητε θάνατόν νύ τοι ὄρκι' ἔταμνον
no consistent reading of Pharr's rules can scan it, "*Homer was wrong*" :P

What about the remaining 12 verses?

- not logical gaps: verses flagged as metrically anomalous in the literature
- e.g. Δ 155: φίλε κασίγνητε θάνατόν νύ τοι ὄρκι' ἔταμνον
no consistent reading of Pharr's rules can scan it, "*Homer was wrong*" :P
- a formal system whose failures are philologically informative

What about the remaining 12 verses?

- not logical gaps: verses flagged as metrically anomalous in the literature



SKILLSCAST

Is a type a lifebuoy or a lamp?

haskellx

haskell

The website

Every verse, explained, Hilbert style!

1 $\overset{-}{\mu}\overset{\sim}{\eta}\overset{\sim}{\nu}\overset{-}{\iota}\overset{\sim}{\alpha}\overset{-}{\epsilon}\overset{-}{\iota}\overset{-}{\delta}\overset{-}{\epsilon}\overset{-}{\theta}\overset{-}{\epsilon}\overset{-}{\alpha}\overset{-}{\Pi}\overset{-}{\eta}\overset{-}{\lambda}\overset{-}{\eta}\overset{-}{\iota}\overset{-}{\acute{\alpha}}\overset{-}{\delta}\overset{-}{\epsilon}\overset{-}{\omega}\overset{-}{\text{A}}\overset{-}{\chi}\overset{-}{\iota}\overset{-}{\lambda}\overset{-}{\eta}\overset{-}{\omicron}\overset{-}{\varsigma}$

(1) $\mu\eta$ is long by nature: it contains the long vowel η . [519]

(2) $\nu\iota$ is short: it is the ultima of $\mu\eta\nu\iota\nu$, whose penult $\mu\eta$ bears the circumflex. [545]

(3) $\epsilon\iota$ is long by nature: it contains the diphthong $\epsilon\iota$. [521]

(4) $\delta\epsilon$ of $\acute{\alpha}\epsilon\iota\delta\epsilon$ is short: its ϵ is short by nature. [519]

(5) $\theta\epsilon$ is short: its ϵ is short by nature. [519]

(6) $\Pi\eta$ is long by nature: it contains the long vowel η . [519]

(7) $\lambda\eta$ is long by nature: it contains the long vowel η . [519]

(8) $\delta\epsilon$ and ω are read as the one syllable $\delta\epsilon\omega$, which counts long. [586]

(9) $\lambda\eta$ is long by nature: it contains the long vowel η . [519]

(10) $\omicron\varsigma$ is short: its $\omicron$ is short by nature. [519]

(11) $\omicron\varsigma$ would be short (10), but counts long: it is the last syllable of the verse. [1184]

(12) $\acute{\alpha}$ must be short to complete the metre. [1169]

(13) $\acute{\alpha}$ must be long to complete the metre. [1169]

(14) ι must be short to complete the metre. [1169]

(15) $\acute{\alpha}$ must be short to complete the metre. [1169]

(16) A must be short to complete the metre. [1169]

(17) $\chi\iota$ must be short to complete the metre. [1169]

Conclusion

We formalised Homeric prosody:

- Pharr's textbook as **stratified inference rules**, mechanised in Agda;
- **sound & complete** decision procedures = scansion with proofs;
- already **found faults** in `hypotactic.com` (e.g. l 382)
- a website **explaining (almost) every verse of the Iliad**, a first.

Conclusion

We formalised Homeric prosody:

- Pharr's textbook as **stratified inference rules**, mechanised in Agda;
- **sound & complete** decision procedures = scansion with proofs;
- already **found faults** in `hypotactic.com` (e.g. l 382)
- a website **explaining (almost) every verse of the Iliad**, a first.

Future work

- other works: Odyssey, Ovid, **Louis de Góngora (11-syllable)**
- beyond prosody: cover **syllabification** as well
 - towards a formal Ancient Greek grammar :O
- integrate into **educational tools**

Conclusion

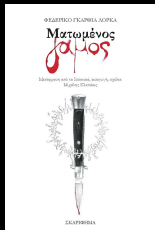
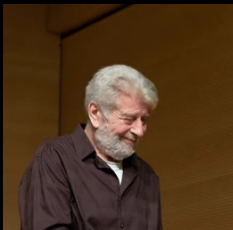
We formalised Homeric prosody:

- Pharr's textbook as stratified in
- sound & complete decision proc
- already found faults in hypotac
- a website explaining (almost) every verse of the Iliad, a first.

Future work

- other works: Odyssey, Ovid, **Louis de Góngora (11-syllable)**
- beyond prosody: cover syllabification as well
 - towards a formal Ancient Greek grammar :O
- integrate into educational tools

Shout-out to **Michalis Klapakis**, my godfather and philologist extraordinaire!



Educational (for us)

- a positivist who dodged Ancient Greek at school...
 - ...now scans Homer **by hand**, decently fast
 - *~~practice~~ rule-debugging makes perfect*

Reflection

Educational (for us)

- a positivist who dodged Ancient Greek at school...
 - ...now scans Homer **by hand**, decently fast
 - *“~~practice~~ rule-debugging makes perfect”*

Educational (for others)

- formalisation + website = study resource of **unprecedented rigour**
- every claim backed by a **machine-checked derivation**

Reflection

Educational (for us)

- a positivist who dodged Ancient Greek at school...
 - ...now scans Homer **by hand**, decently fast
 - “*practice rule-debugging makes perfect*”

Educational (for others)

- formalisation + website = study resource of **unprecedented rigour**
- every claim backed by a **machine-checked derivation**

The Conquest of Bread(th)

- push formal methods beyond math/software
- towards Bogdanov's *Proletkult*: organise all of science (*tektology*) 

Questions



<https://omelkonian.github.io/iliagda/>